

Network Scanning System - Complete Tech Stack & Workflow Guide

1. Overview

This document outlines the complete technology stack and internal workflow of the Python-based Network Scanning System. The system is designed as an Nmap-inspired network discovery and port-scanning tool built entirely in Python, utilizing asynchronous I/O and concurrent processing to ensure speed and reliability without requiring raw sockets or root/administrator privileges.

2. Technology Stack

Core Programming Language

- **Python 3.8+:** The entire scanner is written in Python, chosen for its cross-platform compatibility and extensive standard library capabilities.

Standard Libraries Used

- **asyncio:** Drives the high-performance TCP connect scanning, enabling the tool to scan thousands of ports concurrently without thread overhead.
- **socket:** Used for low-level network connections and banner grabbing.
- **subprocess:** Facilitates calling system-level commands (e.g., the native ping utility for ICMP discovery) while avoiding the need for raw sockets that require root access.
- **ipaddress:** Validates and parses IP addresses, subnets (CIDR), and IP ranges.
- **argparse:** Parses command-line interfaces and scan profiles.
- **re:** Regular expressions are used for parsing target input, extracting version strings from service banners, and processing command outputs.

Third-Party Dependencies

- **rich (>=13.0.0):** Used extensively for generating the styled, colored terminal dashboard, displaying results in readable tables, and managing UI panels.
- **netifaces (>=0.11.0):** Provides accurate detection of local network interfaces, gateways, and subnets. It allows the scanner to dynamically identify the local network segment for "home network" interactive scans.

Operating System Interfacing

- **Windows:** Utilizes built-in ping.exe and arp -a.
 - **Linux:** Uses system ping binary (typically setuid or SOCK_DGRAM) and standard arp routing tables.
 - **macOS:** Uses system ping and arp -a / route -n get default.
-

3. System Architecture & Workflow

The internal pipeline of the scanner is designed as a series of cascading stages. Each stage refines the target data and only passes successfully identified entities to the next stage, saving bandwidth and time.

Stage 1: Target Validation & Parsing

- **Action:** The system parses user input from the CLI (--ports, targets, CIDR notation) or natural language prompts (e.g., "scan my home network").
- **Resolution:** Converts subnets and hostnames into a distinct list of target IP addresses to be scanned.

Stage 2: Host Discovery (Ping Sweep)

- **Primary Method (ICMP):** Uses a subprocess wrapper around the OS's native ping utility to verify if a host is active.
- **Fallback Method (TCP Probe):** If ICMP is blocked, the scanner performs a lightweight TCP probe against common ports (e.g., 80, 443) to check for a response.
- **Enrichment (ARP):** For hosts on the local network segment, ARP-table lookups are performed to resolve MAC addresses.

Stage 3: Port Scanning

- **TCP Scanning (asyncio):** For targets confirmed "UP", the scanner uses Python's `asyncio.open_connection` bounded by a concurrency semaphore (default 100 workers) to perform TCP Connect scans. States are classified as open, closed, or filtered.
- **UDP Scanning:** If requested (`--udp`), a thread-pool scans UDP ports by sending generic payloads and waiting for responses or ICMP unreachable errors.

Stage 4: Service & Version Detection

- **Banner Grabbing:** The system connects to ports identified as open and reads initial socket banners to identify the running service (e.g., SSH, FTP, SMTP).
- **HTTP Probing:** If the port appears to host a web server, a minimal HTTP GET request is sent to retrieve the server headers, status code, and HTML title.
- **Version Matching:** Banners and headers are parsed to extract application versions.

Stage 5: OS Fingerprinting (Passive)

- **Heuristics:** Uses the Time-To-Live (TTL) value from ping responses combined with the footprint of specific open indicator ports (e.g., 3389 for Windows, 22 for Linux).
- **Confidence Scoring:** Returns an OS guess (e.g., "Likely Windows") along with a confidence rating (Low, Medium, High). It operates strictly passively and does not send malicious malformed packets.

Stage 6: MAC & Vendor Lookup

- **OUI Database:** Maps the first 3 octets of discovered MAC addresses to an internal Organizationally Unique Identifier (OUI) table to identify device manufacturers (e.g., Apple, Intel, Raspberry Pi).

Stage 7: Result Aggregation & Reporting

- **Data Compilation:** Per-host objects are aggregated with all discovered properties (Status, Latency, MAC, OS Guess, Open Ports, Services, Versions).
- **Presentation:** Passed to the `rich` formatter to render the final terminal dashboard.
- **Exporting:** If an `--output` flag was provided, the data is concurrently exported to JSON, CSV, plaintext, or a self-contained HTML report.